

Lab5——DAPP

实验日期：2025 年 12 月 1 日

课程名称：区块链基础及其应用

小组成员：田园（2313660），程伟卿（2311865）

一、实验目的

开发一个去中心化的借贷跟踪应用，实现**无第三方信任的债务记录与循环抵消**，核心是：

- 用 Solidity 编写智能合约，存储债务数据并提供核心交互函数；
- 用 web3.js 编写前端客户端，调用合约函数并处理数据展示 / 计算；
- 实现债务循环自动抵消，简化债务关系，最小化 gas 成本。

二、实验环境

工具 / 组件	安装 / 配置操作	核心作用
Node.js	官网下载 LTS 版本安装	提供 JavaScript 运行环境，支持 npm 包管理
Ganache CLI	启动命令 <code>ganache-cli</code>	本地模拟以太坊节点，默认端口 8545
Remix IDE	浏览器打开 https://remix.ethereum.org	Solidity 合约编写、编译、部署、调试
前端环境	打开代码 <code>index.html</code> ，启用浏览器 JavaScript 控制台	调试客户端代码，查看错误日志
开发工具	VS Code	编辑客户端脚本 <code>script.js</code> 和合约代码

三、实验步骤

（一）配置实验环境

1、安装 Node.js

打开官网 <https://nodejs.org/en/>，下载 **LTS 版本**。输入 `node -v` 和 `npm -v`，显示版本号证明安装成功。

```
C:\Users\mojo>node -v
v24.11.1

C:\Users\mojo>npm -v
11.6.2
```

2、安装并启动 Ganache CLI（本地以太坊节点）

命令行执行安装命令：`npm install -g ganache-cli`

启动 Ganache 节点：`ganache-cli`

- 启动成功后，会显示 10 个测试账户（地址 + 私钥）、默认端口 8545、链 ID 等信息；
- **请勿关闭此命令行窗口**（关闭则节点停止，后续操作失效）。

```
C:\Users\mojo>ganache-cli
Ganache CLI v6.12.2 (ganache-core: 2.13.2)

Available Accounts
=====
(0) 0xa6F0c7Bfe94AdafC78B1EE75643e26F43AfBf47b (100 ETH)
(1) 0x446D4f72f07C0DF4517C024Ea07A594414dB5b33 (100 ETH)
(2) 0x485183cc4Efa54bf437841f0c8142bd03E634A93 (100 ETH)
(3) 0xaf1978c72d32a4881B6f3e85B55beba41F5b8618 (100 ETH)
(4) 0x1B98562e60d81122E9696eD764eFaDa6Fde99ed4 (100 ETH)
(5) 0xA48F5E089ef36e093A70E974215274D82263225E (100 ETH)
(6) 0x97852FCf54CFcca06E822cd927e4246B0f2f8088 (100 ETH)
(7) 0xE277d07D1c7dEC0A2b47EbB7BB535aB529d02Ae7 (100 ETH)
(8) 0x75ccc9F16e1A7EEb94d9079959c650568f0F64f1 (100 ETH)
(9) 0xeAB0d8C66139BBE4B5FDfcfc40FFc15A400db182 (100 ETH)

Private Keys
=====
(0) 0x3b244ef69d14f12c219f5990bfec4ca49baa8eb93802b92d5dbd7e2db5572e15
(1) 0x0ba934966bdc798d4f5406974bc053f476b350e3325f9f86754011576f2e8df4
(2) 0x846d23e4d34d262bdae3aba1646ca0549bcc0e9edd70061c76666bb93f2c7c57
(3) 0x9190edf233a36bb765e4cb4003a95ef41911ab0c4022423c98980f0188ef1f12
(4) 0x97c01ae4c38b0304ca543c78ad665427ec99f52e4d35d908485a333dc198fecb
(5) 0xb263f60bfc10cd09b0fd366d526ccf8567f6ec5f7dc7b057c76d24caa13c4cc4
(6) 0x56af99f17ed4aa5a4044afd6ef8efe8a6623c1529303fd2ee3bf7dc8fc598286
(7) 0x14740e357b09c167a60bf87e8e2e07079b35437f810de7f1c9db9abc4daf06b4
(8) 0x2d0f918a74b8126e0eb3fdb04705f145cdcf98b638f1a96d157f5f0b32f8eb98
(9) 0x8a13795898fcd22a75d9b6d4af2b54b9740b461449e2090a6302770326892ebe

HD wallet
=====
Mnemonic:      during walk tragic sound priority until danger truly vast worth trial rhythm
Base HD Path:  m/44'/60'/0'/0/{account_index}

Gas Price
=====
20000000000
```

Gas Limit

6721975

Call Gas Limit

9007199254740991

Listening on 127.0.0.1:8545

(二) 开发智能合约 (Solidity)

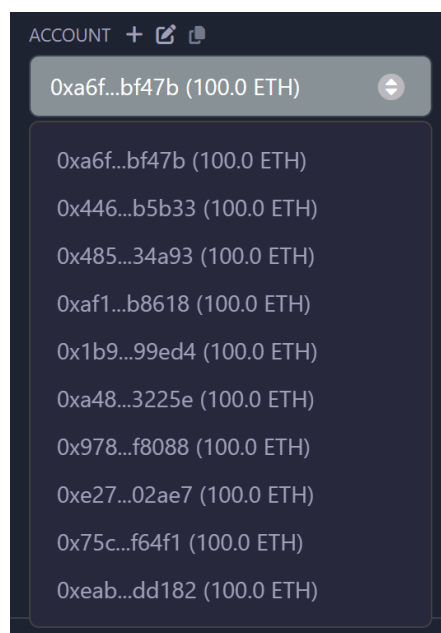
1、配置 Remix IDE

浏览器打开 Remix 官网：<https://remix.ethereum.org>；

创建合约文件：`BlockchainSplitwise.sol`

配置 Remix 连接 Ganache 节点：

- 顶部切换到 “Deploy & Run Transactions” tab；
- “Environment” 下拉选择 **Custom - External Http Provider**；
- 弹出 “External HTTP Provider Endpoint” 输入框，填写 `http://127.0.0.1:8545`；
- 发现 “Account” 面板显示 Ganache 的 10 个测试账户（余额均为100 ETH），证明连接成功。



2、编写 Solidity 合约代码

在 `BlockchainSplitwise.sol` 中编写代码，核心需包含数据存储结构、`lookup` 函数、`add_IOU` 函数

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

contract BlockchainSplitwise {
    // 债务存储: debtor -> creditor -> amount
```

```

mapping(address => mapping(address => uint32)) private _debt;
// 活跃用户列表（避免重复）
address[] private _activeUsers;
mapping(address => bool) private _isActive;
// 用户最后活动时间（秒级时间戳）
mapping(address => uint256) private _lastActive;

function add_IOU(
    address creditor,
    uint32 amount,
    address[] calldata cyclePath,
    uint32 minCycleAmount
) external {
    address debtor = msg.sender;
    require(debtor != creditor, "Debtor and creditor cannot be the same");
    require(creditor != address(0), "Invalid creditor address");
    require(amount > 0, "Amount must be positive");

    // 更新活跃用户和活动时间
    _addActiveUser(debtor);
    _addActiveUser(creditor);
    _lastActive[debtor] = block.timestamp;
    _lastActive[creditor] = block.timestamp;

    // 步骤1: 先添加新债务
    _debt[debtor][creditor] += amount;

    // 步骤2: 循环消解
    if (cyclePath.length > 1 && minCycleAmount > 0) {
        for (uint256 i = 0; i < cyclePath.length - 1; i++) {
            address pathDebtor = cyclePath[i];
            address pathCreditor = cyclePath[i + 1];
            require(_debt[pathDebtor][pathCreditor] >= minCycleAmount, "Insufficient
debt in cycle");
            _debt[pathDebtor][pathCreditor] -= minCycleAmount;
        }
    }
}

function lookup(address debtor, address creditor) external view returns (uint32 ret) {
    return _debt[debtor][creditor];
}

function getActiveUsers() external view returns (address[] memory) {
    return _activeUsers;
}

function getUserLastActive(address user) external view returns (uint256) {
    return _lastActive[user];
}

function _addActiveUser(address user) private {
    if (!_isActive[user]) {

```

```

        _isActive[user] = true;
        _activeUsers.push(user);
    }
}
}

```

合约 `Blockchainsplitwise` 核心目标是 **去中心化债务存储、活跃用户管理、债务循环消解**，所有函数围绕“安全存储债务 + 高效循环抵消”设计，以下是合约的**核心状态变量以及关键函数**：

变量名	类型	核心作用
<code>_debt</code>	<code>mapping(address => mapping(address => uint32))</code>	存储所有债务关系
<code>_activeUsers</code>	<code>address[]</code>	存储活跃用户地址列表
<code>_isActive</code>	<code>mapping(address => bool)</code>	快速检查用户是否活跃
<code>_lastActive</code>	<code>mapping(address => uint256)</code>	记录用户最后活动时间

(1) `add_IOU`（核心业务函数）

- **作用**：外部调用，债务人添加欠条（已有债务则累加），并消解可能的债务循环。
- **书写逻辑**：
 - 取调用者为债务人，用 `require` 校验：非自借贷、债权人地址有效、金额为正；
 - 调用 `_addActiveUser`，将债务人和债权人加入活跃列表，更新二者 `_lastActive` 为当前区块时间；
 - 先在 `_debt` 中累加新债务（避免后续循环消解时金额不足）；
 - 若客户端传入有效循环路径 + 最小金额，遍历路径扣减债务，消解循环；
 - 修饰符 `external`：仅外部调用，高效。
- 循环消解逻辑放在合约层，确保抵消规则不可篡改；用 `uint32` 控制金额范围，节省 gas。

(2) `lookup`（查询函数）

- **作用**：外部查询，获取“债务人欠债权人的具体金额”。
- **书写逻辑**：直接从 `_debt` 双重映射中读值返回，`view` 修饰（仅读不改，无 gas 消耗）。

(3) `getActiveUsers`（辅助查询函数）

- **作用**：外部查询，获取所有活跃用户地址列表。
- **书写逻辑**：直接返回 `_activeUsers` 数组（已去重），`view` 修饰，返回临时副本（防外部修改原数据）。

(4) `getUserLastActive`（辅助查询函数）

- **作用**：外部查询，获取指定用户最后活跃的 UNIX 时间戳。
- **书写逻辑**：读取 `_lastActive` 映射的对应值并返回（`view` 修饰，不消耗 gas）。

(5) `_addActiveUser`（内部辅助函数）

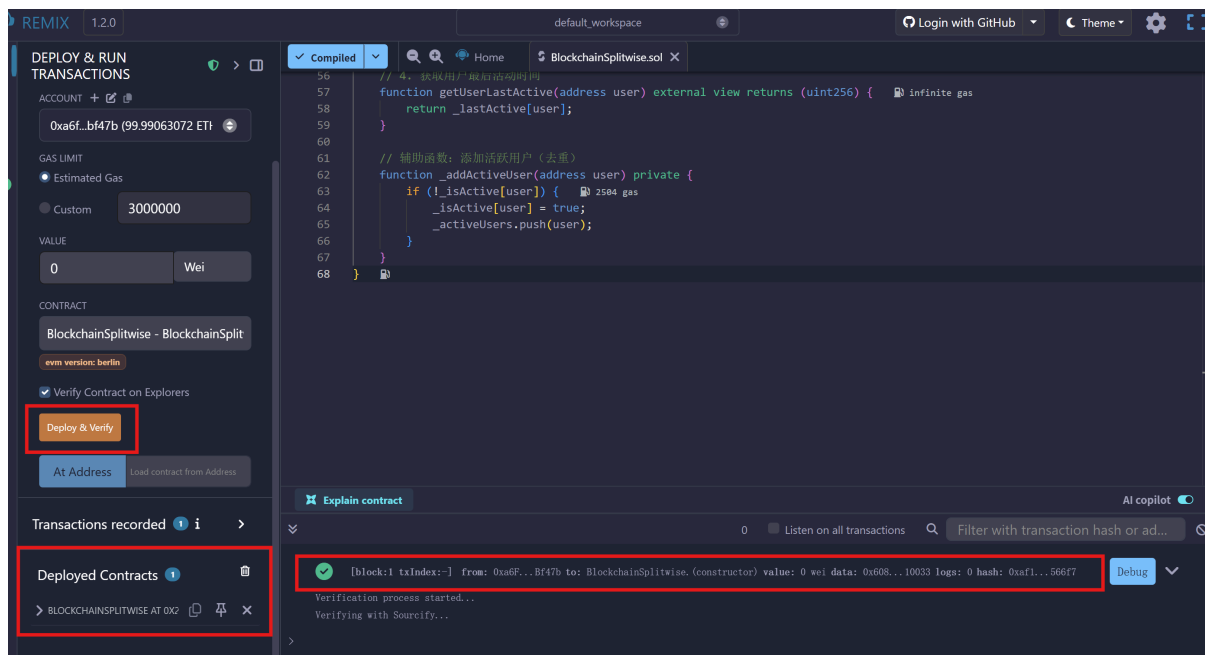
- **作用**：仅内部调用，维护 `_activeUsers` 数组唯一性（防重复，降存储成本）。
- **书写逻辑**：通过 `_isActive` 映射判断用户是否已存在，不存在则添加到 `_activeUsers` 并标记为 `true`（`private` 修饰，仅合约内部调用，避免外部篡改）。

3、编译 Solidity 合约并部署

编译合约，发现无报错信息。点击 **Deploy** 按钮，部署合约到 Ganache 节点，发现**部署成功**：

- 左侧 “Deployed Contracts” 区域显示 BlockchainSplitwise 合约实例；
- 底部交易日志显示 [block:1 txIndex:-] from: 0xa6f...Bf47b to: BlockchainSplitwise，状态为成功；

这意味着合约已被部署到 Ganache 节点的模拟以太坊网络中，可正常调用函数。



4、对接客户端

复制合约信息，将合约地址、ABI填入前端 script.js，实现 DApp 交互。

```
// This is the address of the contract you want to connect to; copy this from Remix
var contractAddress = '0x2F7B6e009bb16AcBa1d8c8f814A64ca63eAf5bb3'

// =====
var abi = [
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "creditor",
        "type": "address"
      },
      {
        "internalType": "uint32",
        "name": "amount",
        "type": "uint32"
      }
    ],
    "name": "addActiveUser",
    "outputs": [],
    "type": "function"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "user",
        "type": "address"
      }
    ],
    "name": "getUserLastActive",
    "outputs": [
      {
        "internalType": "uint256",
        "name": "",
        "type": "uint256"
      }
    ],
    "type": "function"
  }
]
```

```

    },
    {
      "internalType": "uint32",
      "name": "minCycleAmount",
      "type": "uint32"
    }
  ],
  "name": "add_IOU",
  "outputs": [],
  "stateMutability": "nonpayable",
  "type": "function"
},
{
  "inputs": [],
  "name": "getActiveUsers",
  "outputs": [
    {
      "internalType": "address[]",
      "name": "",
      "type": "address[]"
    }
  ],
  "stateMutability": "view",
  "type": "function"
},
{
  "inputs": [
    {
      "internalType": "address",
      "name": "user",
      "type": "address"
    }
  ],
  "name": "getUserLastActive",
  "outputs": [
    {
      "internalType": "uint256",
      "name": "",
      "type": "uint256"
    }
  ],
  "stateMutability": "view",
  "type": "function"
},
{
  "inputs": [
    {
      "internalType": "address",
      "name": "debtor",
      "type": "address"
    },
    {
      "internalType": "address",

```

```

        "name": "creditor",
        "type": "address"
    }
],
"name": "lookup",
"outputs": [
    {
        "internalType": "uint32",
        "name": "ret",
        "type": "uint32"
    }
],
"stateMutability": "view",
"type": "function"
}
];// FIXME: fill this in with your contract's ABI
// =====

```

ABI 包含的 4 个核心函数（与合约功能一一对应）：

函数名	功能	前端调用场景
<code>add_IOU</code>	添加债务并支持循环消解	前端点击“Add IOU”按钮时，调用该函数向合约提交债务数据。
<code>getActiveUsers</code>	获取所有参与 IOU 的用户地址	前端加载时，调用该函数获取用户列表并显示在“All Users”区域。
<code>getUserLastActive</code>	查询用户最后活动时间（时间戳）	前端显示“Last Active”时，调用该函数获取时间并格式化为可读时间。
<code>lookup</code>	查询指定债务人→债权人的债务金额	前端计算“Total Owed”（总欠款）时，调用该函数累加债务金额。

（三）开发客户端（script.js）

1、`getCreditors` 函数

```

// 辅助函数1：获取用户的债权人（供BFS使用）
function getCreditors(user) {
    var creditors = [];
    var allUsers = getUsers();
    for (var i = 0; i < allUsers.length; i++) {
        var potentialCreditor = allUsers[i];
        if (user === potentialCreditor) continue;
        var debt = BlockchainSplitwise.lookup.call(user, potentialCreditor).toNumber();
        if (debt > 0) {
            creditors.push(potentialCreditor);
        }
    }
    return creditors;
}

```


作用：查询指定用户（债务人）的所有债权人（即该用户欠谁钱），为 BFS 循环检测提供“邻居节点”数据；

- 初始化空数组 `creditors` 存储结果；
- 调用 `getUsers()` 获取所有参与借贷的用户，遍历每个用户作为“潜在债权人”；
- 跳过“用户自己”（没人欠自己钱）；
- 调用合约 `lookup` 方法（只读，用 `call`），查询当前用户欠潜在债权人的债务，通过 `toNumber()` 将合约返回的 `BigNumber` 转为整数；
- 若债务金额 > 0 ，说明该潜在债权人是有效债权人，加入 `creditors` 数组；
- 遍历结束后返回 `creditors`。

关键：用 `call()` 调用合约只读函数（不消耗 gas），避免频繁发送交易。

2、getMinDebtOnPath 函数

```
// 辅助函数2：计算路径上的最小债务
function getMinDebtOnPath(path) {
  if (!path || path.length < 2) return 0;
  var minDebt = Infinity;
  for (var i = 0; i < path.length - 1; i++) {
    var debtor = path[i];
    var creditor = path[i + 1];
    var debt = BlockchainSplitwise.lookup.call(debtor, creditor).toNumber();
    if (debt < minDebt) {
      minDebt = debt;
    }
  }
  return minDebt === Infinity ? 0 : minDebt;
}
```

作用：在 BFS 检测到循环路径后，计算路径上所有债务的最小值（循环消解的核心依据）；

- 从合约获取 `getActiveUsers`（合约存储的活跃用户列表），若无则为空数组；
- 调用 `getAllFunctionCalls` 获取所有 `add_IOU` 的历史交易，提取交易发送者（债务人）和参数中的债权人，用 `Set`（自动去重）存储为 `transactionUsers`；
- 合并“合约活跃用户”和“历史交易用户”，用 `Set` 再次去重，避免重复；
- 过滤无效地址（`0x0000000000000000000000000000000000000000`），将有效地址存入 `validUsers`；
- 返回 `validUsers` 数组。

3、getUsers 函数

```
// 获取所有用户
function getUsers() {
  var contractUsers = BlockchainSplitwise.getActiveUsers.call() || [];
  var iouCalls = getAllFunctionCalls(contractAddress, 'add_IOU');
  var transactionUsers = new Set();
  iouCalls.forEach(call => {
    transactionUsers.add(call.from);
    transactionUsers.add(call.args[0]);
  });
}
```

```

});
var allUsers = new Set([...contractUsers, ...transactionUsers]);
var validUsers = [];
allUsers.forEach(user => {
    if (user !== '0x00000000000000000000000000000000000000000000000000000') {
        validUsers.push(user);
    }
});
return validUsers;
}

```

作用：获取所有参与过 IOU 的用户（供前端 “Users” 列表渲染）

- 从合约获取 `getActiveUsers`（合约存储的活跃用户列表），若无则为空数组；
- 调用 `getAllFunctionCalls` 获取所有 `add_IOU` 的历史交易，提取交易发送者（债务人）和参数中的债权人，用 `Set`（自动去重）存储为 `transactionUsers`；
- 合并 “合约活跃用户” 和 “历史交易用户”，用 `Set` 再次去重，避免重复；
- 过滤无效地址（`0x00000000000000000000000000000000000000000000000000000`），将有效地址存入 `validUsers`；
- 返回 `validUsers` 数组。

4、`getTotalOwed` 函数

```

// 计算用户总欠款
function getTotalOwed(user) {
    var total = 0;
    var allUsers = getUsers();
    for (var i = 0; i < allUsers.length; i++) {
        var creditor = allUsers[i];
        if (user === creditor) continue;
        var debt = BlockchainSplitwise.lookup.call(user, creditor).toNumber();
        total += debt;
    }
    return Math.floor(total);
}

```

作用：计算指定用户的总欠款金额（供前端 “Total Owed” 展示）

- 初始化 `total` 为 0（累加容器）；
- 调用 `getUsers()` 获取所有用户，遍历每个用户作为 “潜在债权人”；
- 跳过 “用户自己”；
- 调用合约 `lookup` 查该用户欠潜在债权人的债务，转整数后累加到 `total`；

5、`getLastActive` 函数

```

// 获取用户最后活动时间
function getLastActive(user) {
    var lastActiveTime = null;
    var contractTime = BlockchainSplitwise.getUserLastActive.call(user).toNumber();
    if (contractTime > 0) {

```

```

        lastActiveTime = contractTime;
    }
    var iouCalls = getAllFunctionCalls(contractAddress, 'add_IOU');
    var curBlock = web3.eth.blockNumber;
    while (curBlock !== GENESIS) {
        var block = web3.eth.getBlock(curBlock, true);
        var txns = block.transactions;
        for (var j = 0; j < txns.length; j++) {
            var txn = txns[j];
            if ((txn.from === user || (txn.to === contractAddress && txn.input)) &&
block.timestamp) {
                try {
                    var funcCall = abiDecoder.decodeMethod(txn.input);
                    if (funcCall && funcCall.name === 'add_IOU' && funcCall.params[0].value
=== user) {
                        if (block.timestamp > lastActiveTime) {
                            lastActiveTime = block.timestamp;
                        }
                    }
                } catch (e) {
                    continue;
                }
                if (txn.from === user && block.timestamp > lastActiveTime) {
                    lastActiveTime = block.timestamp;
                }
            }
        }
        curBlock = block.parentHash;
    }
    return lastActiveTime > 0 ? lastActiveTime : null;
}

```

作用：获取用户最后一次参与 IOU 的时间戳（供前端“Last Activity”展示）

- 初始化 lastActiveTime 为 null;
- 先从合约 getUserLastActive 获取存储的时间戳，转整数后若 > 0，更新 lastActiveTime;
- 调用 getAllFunctionCalls 获取所有 add_IOU 历史交易，再从当前区块遍历到创世块（GENESIS），查包含该用户的交易;
- 对每个区块的交易，判断：若交易发送者是该用户，或交易指向合约且是 add_IOU（解码输入确认）且债权人是该用户，更新 lastActiveTime 为区块时间戳（取最大者）;
- 遍历结束后，若 lastActiveTime > 0 则返回，否则返回 null。

6、add_IOU 函数

```

// 添加IOU并自动检测完整循环
function add_IOU(creditor, amount) {
    creditor = creditor.trim();
    amount = parseInt(amount, 10);
    if (isNaN(amount) || amount <= 0) {
        alert("Amount must be a positive number!");
    }
}

```

```

        return;
    }
    var debtor = web3.eth.defaultAccount;
    if (debtor === creditor) {
        alert("Cannot owe money to yourself!");
        return;
    }
    if (creditor === '0x0000000000000000000000000000000000') {
        alert("Invalid creditor address!");
        return;
    }

    // 检测「creditor→debtor」的路径（形成完整循环）
    var cyclePath = doBFS(creditor, debtor, getCreditors);
    var minCycleAmount = 0;
    if (cyclePath && cyclePath.length > 1) {
        // 1. 计算已有路径的最小债务（如A→B→C的最小债务10）
        var existingMinDebt = getMinDebtOnPath(cyclePath);
        // 2. 最小债务 = min(已有路径最小债务, 新IOU金额)（确保不超过新金额）
        minCycleAmount = Math.min(existingMinDebt, amount);
        // 3. 拼接完整循环路径（如[A,B,C]→[A,B,C,A]，包含新边C→A）
        cyclePath = cyclePath.concat([creditor]);
    }

    // 调用合约：先添加新债务，再消解循环
    try {
        BlockchainSplitwise.add_IOU.sendTransaction(
            creditor,
            amount, // 传递原金额
            cyclePath || [],
            minCycleAmount,
            { from: debtor, gas: 600000 } // 提升gas，适配循环消解
        );
    } catch (err) {
        alert("Failed to add IOU: " + err.message);
        console.error("IOU error:", err);
    }
}

```

作用：前端触发添加 IOU 的入口（绑定“Add IOU”按钮点击事件）

- **输入预处理：**清除债权人地址空格，将金额转为整数；
- **输入校验：**
 - 金额非正 / 非数字：弹窗警告并返回；
 - 债务人（默认账户 `web3.eth.defaultAccount`）与债权人相同：弹窗警告（禁止自借）；
 - 债权人是无效地址：弹窗警告；
- **循环检测：**调用 `doBFS` 找“债权人→债务人”的路径（形成完整循环）；
- **循环金额计算：**
 - 若存在有效路径，调用 `getMinDebtOnPath` 算路径最小债务，取“该值与新欠条金额的较小值”（避免扣减超新债务）；

- 拼接完整循环路径（如 [A,B,C] → [A,B,C,A]，包含新边）；
- **调用合约：**用 `sendTransaction` 调用合约 `add_IOU`（写操作，需指定发送者和 gas 上限 600000，适配循环消解）；
- **异常处理：**捕获调用错误，弹窗提示并打印日志。

四、实验验证

1、页面初始化验证

- **My Account 区域：**显示当前默认账户地址，“Total Owed”初始为 \$0（无任何 IOU 时），“Last Activity”显示当前时间；
- **Add IOU 区域：**“Address of person you owe”和“Amount you owe them”输入框为空，“Add IOU”按钮可点击；
- **Users 区域：**初始为空，右下角“Wallet Addresses”会完整列出 Ganache 的所有测试账户地址（10个），无空值或乱码。
- 由此可知，页面初始化验证通过。

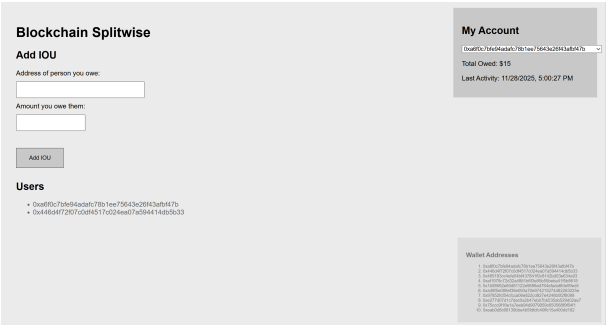


2、循环债务消解功能验证

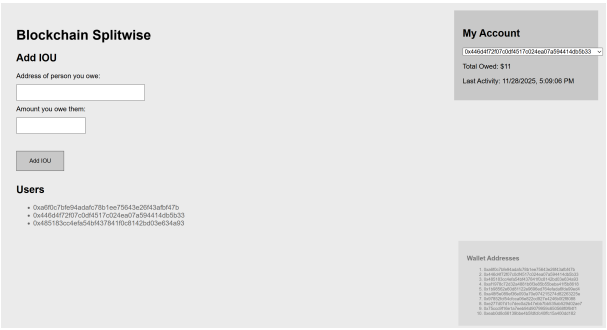
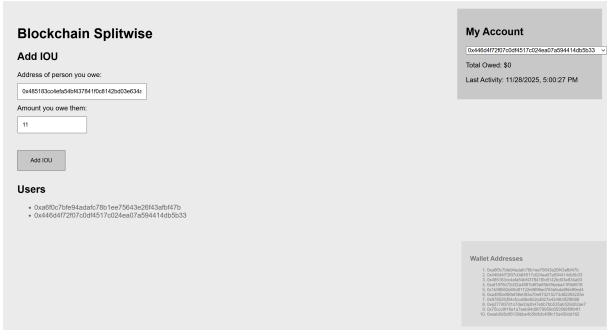
我们主要使用一下三个账户进行循环债务消解功能的验证。

A: 0xa6f0c7bfe94adafc78b1ee75643e26f43afb47b
B: 0x446d4f72f07c0df4517c024ea07a594414db5b33
C: 0x485183cc4efa54bf437841f0c8142bd03e634a93

账户 A 给 B 添加 IOU 15：Users列表新增 A 和 B 账户，添加后 A 的 Total Owed 为15。

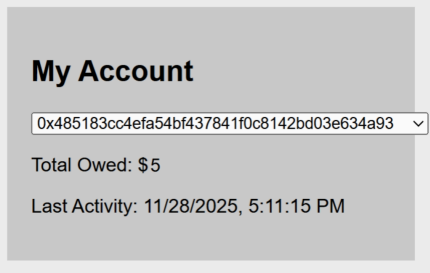
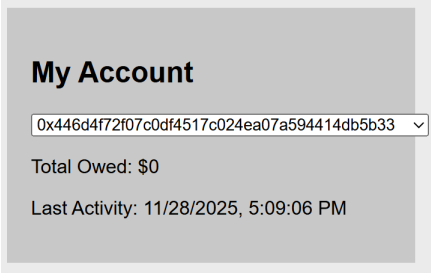
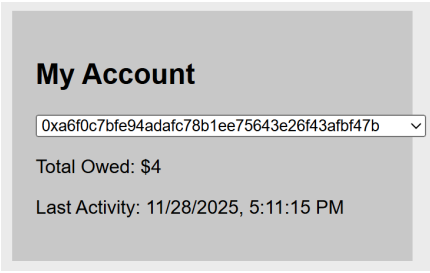


账户 B 给 C 添加 IOU 11：Users列表新增 C 账户，添加后 B 的 Total Owed 为11。



账户 C 给 A 添加 IOU 16：Users列表未新增账户，添加后 A 的 Total Owed 为4，B 的 Total Owed 为0，C 的 Total Owed 为5

项目	A→B	B→C	C→A	说明
初始债务	15	11	16	循环路径 A→B→C→A 的初始债务关系
最小债务金额	11	11	11	$\min(15, 11, 16) = 11$
消解后剩余债务	4	0	5	各路径债务减去最小金额11



五、实验总结

通过本次实验，我系统掌握了区块链 DAPP 开发的核心能力。

技术上，学会用 Solidity编写含债务记录、循环消解的智能合约，如用双重映射存债务、require 校验参数；熟练使用 web3.js 实现前端交互，区分 call() 查数据与 sendTransaction() 改状态，还掌握了 Node.js、Ganache CLI、Remix IDE（合约部署调试）的全流程实操。

开发流程上，理清了“环境搭建→合约开发→前端联调→功能测试”的链路，先在 Remix 获合约地址，再写入前端脚本实现通信。还理解了债务循环消解的实现，通过 BFS 找路径、算最小债务，再调用合约简化债务。